THENI KAMMAVAR SANGAM
COLLEGE OF ARTS AND SCIENCE (Co-Ed)
Approved by Govt. of Tamil Nadu | Affiliated to Madurai Kamaraj University
Recognized under section 2(f) of the UGC Act, 1956 | ISO 9001:2015 Certified institution
KODUVILARPATTI, THENI- 625 534.
04546 - 250882, 8925192579, www.tkscas.com, tksclgas@gmail.com

DOC-GENIE

Automatic Python Docstring Generator

SUBMITTED BY

P. ABINAYA (REG.NO. C3S29951)

S.ANUSIYA (REG.NO. C3S29952)

P. BOWYADHARSHINI (REG.NO C3S29953)

R.BRUNDHA (REG.NO. C3S29954)

S DIVITHIYAMANI (REG.NO C3S29955)

Department of Computer Science

Academic Year

2025 – 2026

TABLE OF CONTENTS

S.No	Content	Page No
1	Abstract	1
2	Introduction	2
3	Objectives of the Project	3
4	Technologies Used	4
5	System Architecture	5
6	Implementation Methodology	6
7	System Features	7
8	Testing and Results	8
9	Future Scope	9
10	Conclusion	10

ABSTRACT

Doc-Genie is an intelligent tool designed to automatically generate Python docstrings for functions. Writing proper documentation for code is an important part of software development, but many programmers skip it because it takes time and effort.

This project simplifies the documentation process by allowing users to paste Python functions into the system. The tool analyzes the code and automatically generates structured docstrings that explain the function, its parameters, and return values.

The system uses Python's Abstract Syntax Tree (AST) module to analyze the structure of the code and understand function definitions, parameters, return types, loops, and conditional logic.

A user-friendly interface is built using Gradio so users can easily input their Python code and instantly receive generated documentation.

The system also allows exporting the generated documentation as a PDF file using ReportLab. This project demonstrates how automation can improve code documentation and help developers maintain clean and understandable code.

INTRODUCTION

In software development, documentation plays an important role in making code understandable and maintainable. One of the most common forms of documentation in Python programs is the docstring.

A docstring is a string written inside a function that explains what the function does, the parameters it accepts, and the value it returns.

However, many developers do not write proper docstrings because it requires additional effort and time.

The **Doc-Genie project** is designed to solve this problem by automatically generating docstrings for Python functions.

The system analyzes the code using Python's AST module and identifies the function name, parameters, return type, and logical operations such as loops and conditions.

Based on this information, the system generates structured docstrings in different formats such as **Google style** and **NumPy style**.

This project helps developers quickly generate documentation and maintain well-documented code.

OBJECTIVES OF THE PROJECT

The main objectives of this project are:

- To automatically generate docstrings for Python functions
- To reduce the effort required for manual code documentation
- To analyze Python code structure using AST
- To support multiple documentation styles (Google and NumPy)
- To provide an interactive user interface using Gradio
- To allow exporting generated documentation as a PDF file

- To automatically generate docstrings for Python functions based on the function structure.

- To reduce the time and effort required for writing manual documentation.

- To analyze Python code using the **Abstract Syntax Tree (AST)** and extract important details such as function name, parameters, and return values.

- To support multiple documentation styles such as **Google style** and **NumPy style**.

- To provide a simple and interactive user interface using **Gradio** for easy code input and output visualization.

- To improve code readability and maintainability through proper documentation.

- To automatically format Python code using **Autopep8** according to standard coding conventions.

- To detect logical elements such as loops, conditions, and arithmetic operations in the code.

- To generate clear explanations for the functionality of the Python function.

- To allow users to export the generated documentation as a **PDF file**.

TECHNOLOGIES USED

Python

Python is the main programming language used to develop the project and implement the analysis logic.

AST (Abstract Syntax Tree)

The AST module is used to analyze Python code and extract function details such as parameters and return types.

Gradio

Gradio is used to build the web-based interface that allows users to input Python code and generate docstrings interactively.

Autopep8

Autopep8 is used to automatically format Python code according to standard coding style guidelines.

ReportLab

ReportLab is used to generate PDF files containing the generated documentation.

Google Colab

Google Colab provides a cloud-based development environment where the project can be executed without local installation.

SYSTEM ARCHITECTURE

The Doc-Genie system consists of several modules working together.

User Interface Module

The user enters Python code using the Gradio interface.

Code Analysis Module

The system analyzes the code using Python's AST module to extract function details.

Logic Detection Module

This module checks whether the function contains loops, conditions, or arithmetic operations.

Docstring Generation Module

Based on the analysis, the system generates structured docstrings in either Google style or NumPy style.

Code Formatting Module

Autopep8 is used to format the code according to Python coding standards.

Export Module

The generated documentation can be exported as a PDF file using ReportLab.

IMPLEMENTATION METHODOLOGY

The implementation of the project involves the following steps.

Environment Setup

Google Colab is used to install required libraries such as Gradio, ReportLab, and Autopep8.

Code Parsing

The Python code entered by the user is parsed using the AST module to detect function definitions.

Function Analysis

The system analyzes the function parameters, return values, loops, and conditions.

Docstring Generation

Based on the analysis, the system generates a structured docstring in the selected style.

Code Formatting

Autopep8 is used to improve code readability and ensure proper formatting.

Interface Development

The Gradio framework is used to build an interactive interface for code input and output display.

PDF Export

ReportLab is used to generate a PDF containing the final documented code.

SYSTEM FEATURES

The Doc-Genie system provides the following features:

- Automatic Python docstring generation
- Support for Google and NumPy docstring styles
- Intelligent code analysis using AST
- Automatic code formatting using Autopep8
- Interactive web interface using Gradio
- PDF export of generated documentation

TESTING AND RESULTS

Test Case 1

Input Function:

```
def add(a, b):
    return a + b
```

Generated Docstring:

Add.

Performs computation. Applies addition.

Args:
    a (Any): Description of a.
    b (Any): Description of b.

Returns:
    Any: Description of return value.

Generated Description:

The system detects conditional logic and generates a corresponding explanation.

The testing results show that the system successfully analyzes Python functions and generates meaningful docstrings.

FUTURE SCOPE

The system can be improved with several additional features:

- Support for class and method documentation
- Integration with GitHub repositories
- Support for more docstring formats such as Sphinx
- AI-based description generation for better explanations
- Support for multiple programming languages

CONCLUSION

Doc-Genie is an intelligent tool designed to automatically generate documentation for Python functions.

By analyzing code structure using the AST module, the system can detect function details and generate structured docstrings automatically.

The integration of Gradio provides a simple and interactive interface, while the PDF export feature allows users to save documentation easily.

This project demonstrates how automation can improve code documentation and help developers maintain clean, understandable, and well-documented programs.